

Lec6 回顾: 过程 (Procedures)

ICS小班回课

信息科学技术学院 王度 2025-10-15

目标

- 理解 `call/ret` 和 `push/pop` 这两对核心指令的功能。
- 理解程序栈的结构，能够识别栈上的不同组成部分：
 - 返回地址
 - 函数参数
 - 保存的寄存器
 - 局部变量
- 解释 **调用者保存** 和 **被调用者保存** 寄存器的区别。
- 阐述栈结构如何自然地支持函数 **递归**。

过程调用的三大机制

当一个过程 `P` 调用另一个过程 `Q` 时，需要一系列机制来协同工作。

传递控制

- 程序计数器(`%rip`)需要能从 `P` 跳转到 `Q` 的起始处。
- `Q` 执行完毕后，要能返回到 `P` 调用 `Q` 之后的那条指令。

传递数据

- `P` 需要能向 `Q` 传递一个或多个参数。
- `Q` 需要能向 `P` 返回一个值。

内存管理

- `Q` 在执行期间需要空间来存储其局部变量。
- `Q` 返回时，这部分空间必须被释放。

这些机制的具体实现由应用二进制接口 (Application Binary Interface, ABI) 规定，ABI是一种人工设计的交互规范。

x86-64 栈结构

栈是内存中一块特殊的区域，遵循“后进先出”(LIFO)原则。

- **增长方向:** x86-64中，栈向 **低地址** 方向增长。
- **栈顶指针:** `%rsp` 寄存器始终指向栈顶（当前栈上最低的字节的地址）。

栈操作: pushq & popq

pushq 和 popq 是操作栈的两个基本指令。

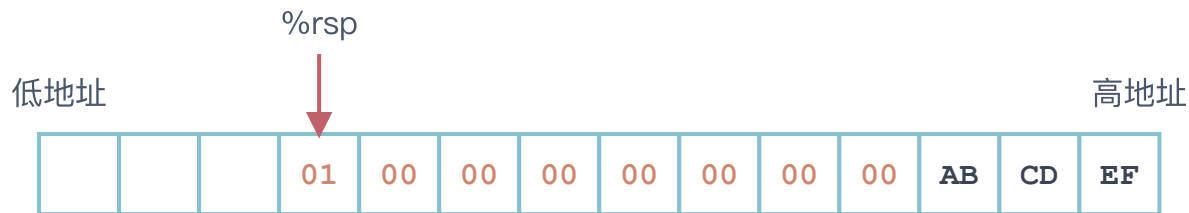
- pushq val (入栈)
 1. 读取 val 的值。
 2. %rsp 的值减 8。
 3. 将读取到的 val 存入新的 %rsp 指向的内存地址。
- popq dest (出栈)
 1. 从 %rsp 指向的地址读取值。
 2. %rsp 的值加 8。
 3. 将读取到的值写入 dest 。

popq 只是移动 %rsp，内存中的数据并未被立即清除，只是变得无效了。

注意顺序！

- pushq 先读取值，移动 %rsp，再存值。
 - pushq %rsp时，存入栈中的值是原先%rsp指向的地址。
- popq 先读取值，再移动 %rsp，再写入dest。

pushq \$0x1 可视化 (小端法)



调用约定 I: 控制流

`call` 和 `ret` 指令利用栈来实现过程调用的控制权转移。

- `call Label / call *Operand`

1. **压栈**: 将 **返回地址** (注意是紧跟在 `call` 后面的那条指令的地址而不是 `call` 指令的地址) 压入栈顶。
2. **跳转**: 将 `%rip` (程序计数器) 更新为 `Label` 或 `*Operand` 的地址, 开始执行被调用函数。

- `ret`

1. **出栈**: 从栈顶弹出一个地址, 该地址是之前 `call` 指令存入的返回地址。
2. **跳转**: 将 `%rip` 更新为这个返回地址, 回到调用者继续执行。

调用约定 II: 数据流

函数参数和返回值如何传递？

参数传递

前 6 个整数或指针参数通过寄存器传递：`%rdi`
`%rsi` `%rdx` `%rcx` `%r8` `%r9`

第 7 个及以后的参数通过 **栈** 传递。

通过栈传递参数时，所有的数据大小都向8字节的倍数对齐。例如：

```
1 void foo( ... , int a, char b, double c);
```

调用时如果通过栈传 `a` , `b` , `c` :

`a` → 8字节 (虽然是4字节) `b` → 8字节 (虽然是1字节) `c` → 占8字节 (本身就是8字节)

返回值

通常使用 `%rax` 寄存器来存放返回值。

示例: `long result = func(a, b);`

- `a` 会被放入 `%rdi`
- `b` 会被放入 `%rsi`
- `func` 执行完后, `caller` 从 `%rax` 中读取 `result` 。

栈帧 (Stack Frames)

作用: 保存一次函数调用所需的所有 状态信息。

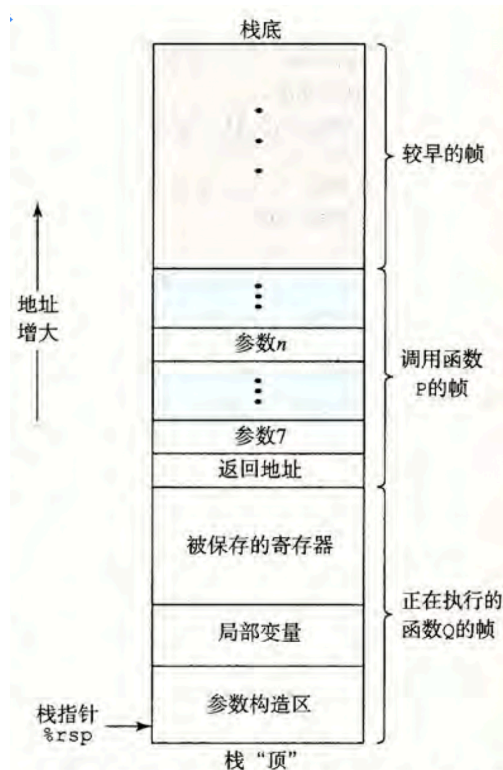
内容:

- 参数7~参数n (属于P的栈帧)

注意顺序! 第一个弹出的是参数7, 以此类推

- 返回地址 (属于P的栈帧)
- 需要保存的寄存器值 (被调用者保存寄存器) (属于Q的栈帧)
- 局部变量 (属于Q的栈帧)
- Q调用其他函数时, 需要构造的参数 (属于Q的栈帧)

栈帧机制使得函数可以重入 (Reentrant), 是实现递归的基础。%rbp: 帧指针, 有助于访问栈帧中的变量。也是一个被调用者保存寄存器。



栈帧16字节对齐

- ABI 规定，进入被调用函数时（call前），%rsp 必须是 16 字节对齐。
- 执行 call 时，压入返回地址（8 字节），导致 %rsp 变成 $16N-8$ 的形式。
- 因此此后再调用其他函数时，必须保证再call之前%rsp仍然是 16 字节对齐的，这可以通过保存寄存器、分配空间、在调用前手动调整 %rsp 等手段来实现。

帧指针 (%rbp) 的使用是可选的

```
1  # 有 %rbp 的函数栈帧
2  push %rbp
3  mov %rsp, %rbp
4  sub $16, %rsp  # 分配局部变量空间
5  ...
6  mov %rbp, %rsp
7  pop %rbp
8  ret
```

```
1  # 省略 %rbp 的函数
2
3
4  sub $16, %rsp
5  ...
6  add $16, %rsp
7
8  ret
```

局部变量的保存

```
1  long bar(long* x1, long* x2, long* x3, long* x4,  
2      long* x5, long* x6, long* x7, long* x8)  
3  {  
4      return *x1 + *x2 + *x3 + *x4 + *x5 + *x6 + *x7 + *x8;  
5  }  
6  long foo() {  
7      long x1 = 1, x2 = 2, x3 = 3, x4 = 4;  
8      long x5 = 5, x6 = 6, x7 = 7, x8 = 8;  
9      return bar(&x1, &x2, &x3, &x4, &x5, &x6, &x7, &x8);  
10 }
```

局部变量不需要按照8字节对齐。

注意局部变量的生长方向与参数的生长方向是相反的：

```
1  foo():  
2      subq    $64, %rsp  
3      movq    $1, 56(%rsp)  
4      movq    $2, 48(%rsp)  
5      movq    $3, 40(%rsp)  
6      movq    $4, 32(%rsp)  
7      movq    $5, 24(%rsp)  
8      movq    $6, 16(%rsp)  
9      movq    $7, 8(%rsp)  
10     movq    $8, (%rsp)  
11     leaq    32(%rsp), %rcx  
12     leaq    40(%rsp), %rdx  
13     leaq    48(%rsp), %rsi  
14     leaq    56(%rsp), %rdi  
15     movq    %rsp, %rax    # 8  
16     pushq   %rax  
17     leaq    16(%rsp), %rax # 7(%rsp变化了)  
18     pushq   %rax  
19     leaq    32(%rsp), %r9  
20     leaq    40(%rsp), %r8  
21     call    bar  
22     addq    $80, %rsp  
23     ret
```

一个潜在的问题

如果调用者(caller)和被调用者(callee)都想用同一个寄存器...

```
1  // yoo 是 caller, who 是 callee
2  void yoo() {
3      long val = 15213; // 假设 val 存在 %rdx
4      who();
5      // 此时 %rdx 的值还是 15213 吗?
6      long result = val + ...;
7  }
8
9  void who() {
10     long temp = 18213;
11     // ... who() 内部也用了 %rdx ...
12 }
```

问题: who 可能会修改 %rdx , 破坏 yoo 的数据。

解决方案: 必须有一个清晰的 寄存器保存约定。

寄存器保存约定

x86-64 ABI 将寄存器分为两类：

调用者/保存 (Caller-Saved)

意思：寄存器由 调用者 负责保存，被调用者可以随意使用。

- **规则:** 如果 调用者 希望在 被调用者 返回后，某个寄存器的值保持不变，调用者 必须在 `call` 指令之前 自己保存好这个寄存器的值 (例如压入栈)。
- **视角:** 调用者 认为这些寄存器是“易变的”。

被调用者/保存 (Callee-Saved)

意思：寄存器由 被调用者 负责保存，调用者可以放心使用。

- **规则:** 如果 被调用者 想要使用这类寄存器，它 必须先保存 其原始值，并在 返回 调用者 之前恢复它。
- **视角:** 调用者 可以放心地认为这些寄存器在函数调用过程中是“不变的”。

x86-64 寄存器约定分类

Callee-Saved (被调用者保存)

`callee` 使用前必须保存，返回前必须恢复。

- `%rbx`
- `%rbp` (可选的帧指针)
- `%r12` , `%r13` , `%r14` , `%r15`

记忆: `%rbx` 和 `%rbp` 都含有字母 b (backup)

Caller-Saved (调用者保存)

`caller` 自己负责保存。`callee` 可以随意使用。

- 其余除去`%rsp`的所有寄存器

例子分析

```
1  long incr(long *p, long val) {
2      long x = *p;
3      long y = x + val;
4      *p = y;
5      return x;
6  }
7
8  long call_incr(long x) { // x 在 %rdi
9      long v1 = 15213;
10     long v2 = incr(&v1, 3000); // call incr 会覆盖 %rdi
11     return x + v2;
12 }
```

常见的必须将局部数据存储在内存中的情况：寄存器不够、使用了
&运算符、使用了数组或结构 (CSAPP:170)

```
1  incr(long*, long):
2      movq    (%rdi), %rax
3      addq    %rax, %rsi
4      movq    %rsi, (%rdi)
5      ret
6  call_incr(long):
7      pushq   %rbx
8      subq    $16, %rsp
9      movq    %rdi, %rbx
10     movq    $15213, 8(%rsp)
11     leaq    8(%rsp), %rdi
12     movl    $3000, %esi
13     call    incr(long*, long)
14     addq    %rbx, %rax
15     addq    $16, %rsp
16     popq    %rbx
17     ret
```


递归是如何工作的？

无需任何特殊处理！

递归调用本质上就是函数自己调用自己。标准的调用机制完全适用。

1. **新的栈帧**: 每次递归调用都会创建一个 **新的、独立的栈帧**，用于存放该层调用的局部变量、参数和返回地址。
2. **数据隔离**: 不同层级的调用有各自的栈帧，因此它们的局部变量互不干扰。
3. **寄存器保存**: `Caller/Callee-Saved` 约定依然有效，保证跨层调用时寄存器使用的正确性。

当递归到达**终止条件**并开始返回时，栈帧会**逐层销毁**，程序也逐层返回。

总结

- **栈**: 实现过程调用的核心数据结构 (LIFO)。
- **调用约定 (ABI)**: 统一了控制流、数据流和内存管理的规则，是不同模块协作的基础。
- **栈帧**: 为每次函数调用提供了独立的“工作区”，隔离了状态。
- **寄存器保存约定**: 通过 `Caller-Saved` 和 `Callee-Saved` 规则，优雅地解决了寄存器复用时的冲突问题。
- **递归**: 在上述机制下被自然而然地支持，无需额外处理。